

Chapter 9. API Gateway

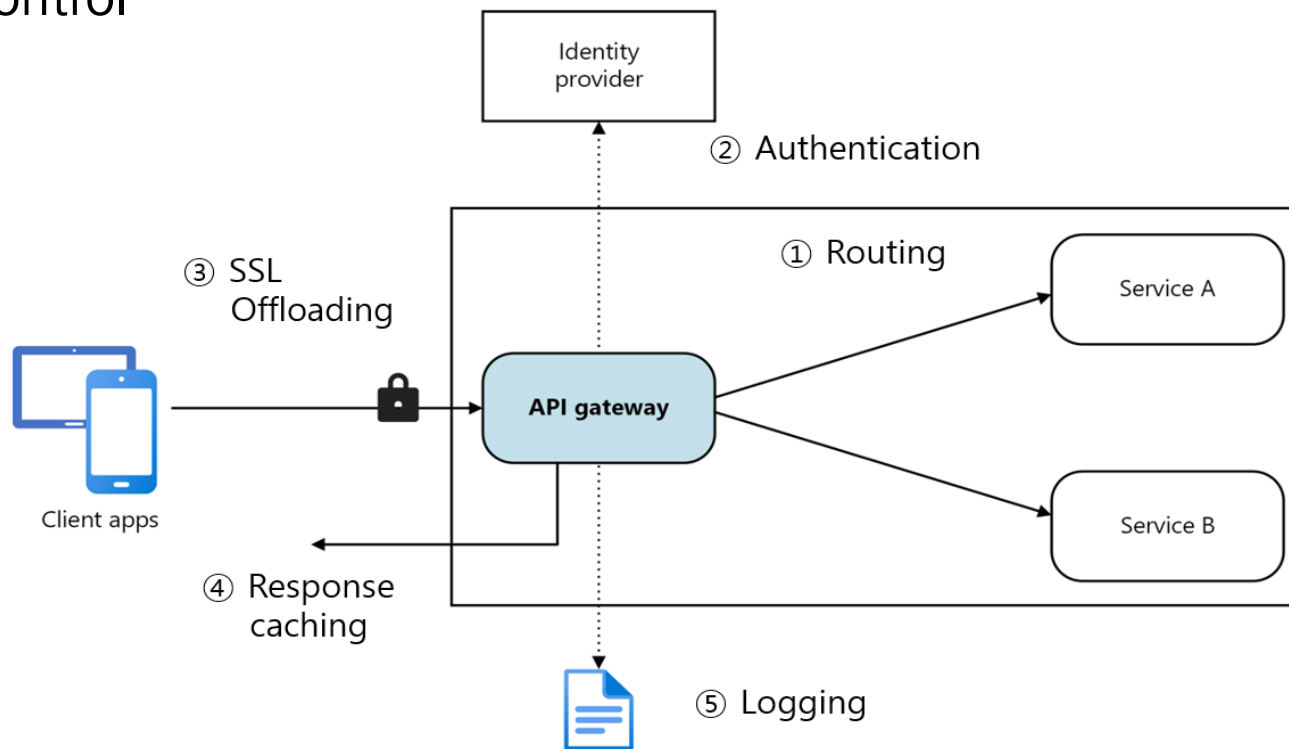


Lee youngkon



Basic concept

- ✓ API Gateway is a single entry point between the client and the microservice 📱 Edge server
- ✓ Receive all requests and route them to the appropriate service
- ✓ Common features such as authentication, logging, monitoring, and traffic control



Problems with direct access to services without G/W

- ✓ Client calls multiple services directly
- ✓ Client modification required when service changes
- ✓ Implement authentication/logging redundancy
- ✓ Increased network complexity

Gateway Benefit

- ✓ Provides a single entry point
- ✓ Centralize common features
- ✓ Simplifying the Client
- ✓ Enhanced Security

1. Routing

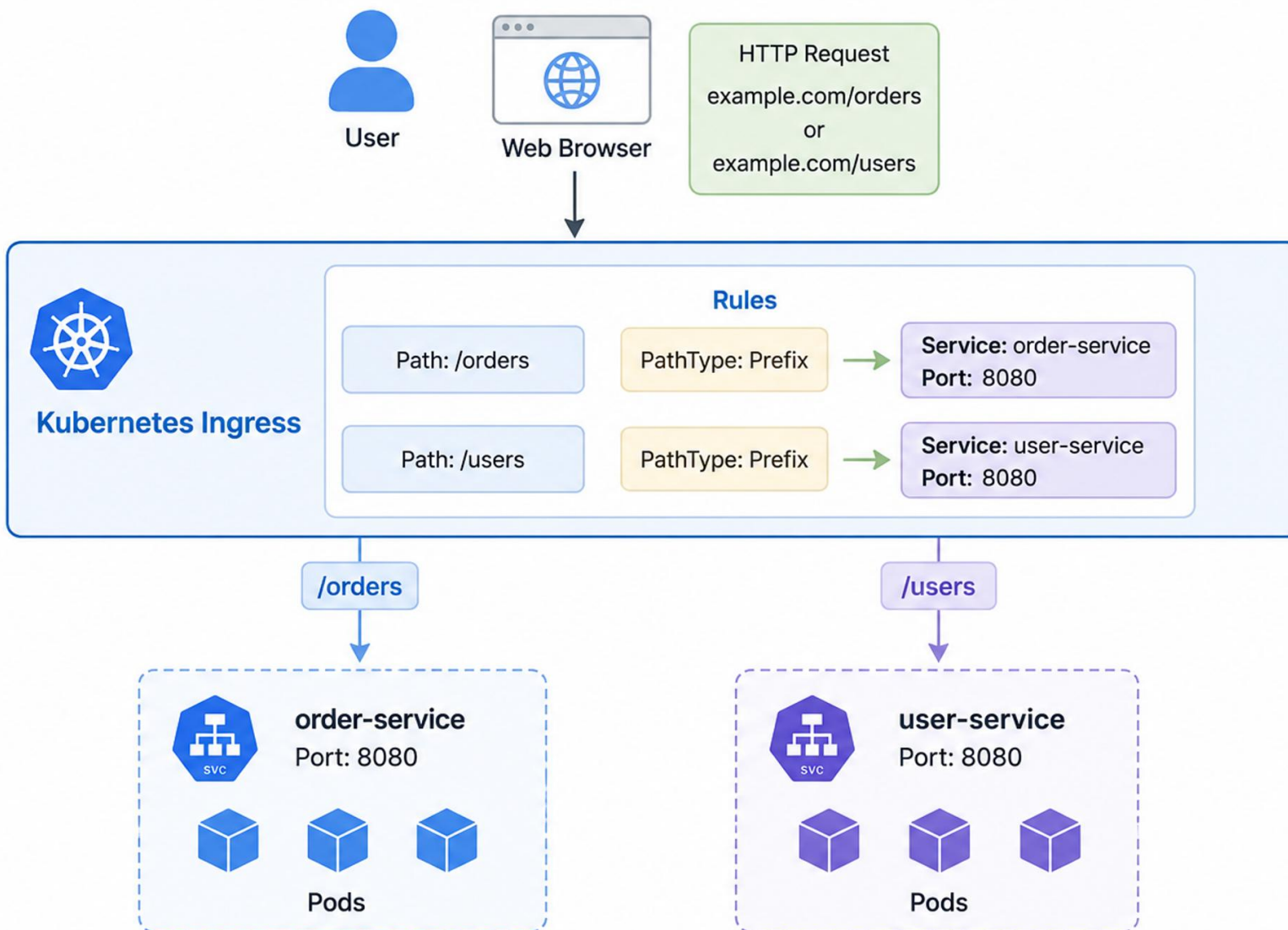
- ✓ Path, Host, Header-based Routing
- ✓ Static Routing vs Dynamic Routing (Service Discovery associated)
- ✓ Rewrite, PrefixStrip, Path convert
- ✓ Canary / Blue-Green Routing

Routing methods: Path-based routing

- ✓ How to forward to different services depending on URL path
 - /api/orders/** → order-service
 - /api/users/** → user-service
 - /static/** → static-server
- ✓ Characteristics
 - Used mostly
 - Fits well with REST API structure
 - API Gateway, Ingress, Service Mesh supporting

01 API G/W Key Features

Routing methods: Path-based routing



Routing methods: Path-based routing

```
rules:
- http:
  paths:
  - path: /orders
    pathType: Prefix
    backend:
      service:
        name: order-service
        port:
          number: 8080
  - path: /users
    pathType: Prefix
    backend:
      service:
        name: user-service
        port:
          number: 8080
```

Routing methods: host-based routing

- ✓ Service branch according to domain (hostname)
- ✓ E.g.
 - `api.company.com` → `api-service`
 - `admin.company.com` → `admin-service`
 - `shop.company.com` → `shop-service`
- ✓ Characteristics
 - Domain separation by service is possible
 - Frequently used in SaaS multi-tenant structures
 - associated with HTTPS certificate management

EXAMPLE

```
tls:  
- hosts:  
  - "*.example.com"  
secretName: wildcard-cert
```

Routing methods: host-based routing

```
rules:
- host: api.company.com
  http:
    paths:
    - path: /
      backend:
        service:
          name: api-service
- host: admin.company.com
  http:
    paths:
    - path: /
      backend:
        service:
          name: admin-service
```

Routing methods: header-based routing

✓ Viewing HTTP Header Values and Routing

- E.g : X-Version: v2

✓ A/B test

- Only some users use the new version

✓ Internal Administrator Features

- X-Admin: true

✓ Mobile/Web Separation

- User-Agent: Mobile

http:

– match:

– headers:

x-version:

exact: v2

route:

– destination:

host: product-service

subset: v2

Routing methods: static routing

✓ Routing destination fixed

✓ Characteristics

- Simple
- Fast
- Difficulty of changing operations
- Server IP changes require modification

```
route:  
  host: 10.0.0.10
```

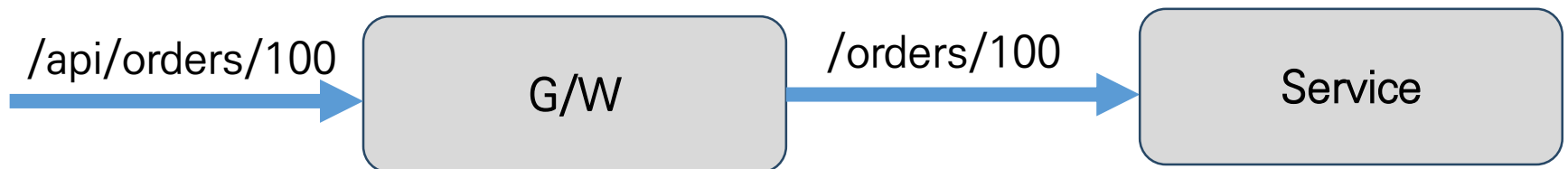
```
backend:  
  service: order-service
```

Routing methods: Dynamic routing

- ✓ Change targets in real time in conjunction with Service Discovery
- ✓ Automatically exclude Pod when it dies
- ✓ Automatically add new pods when created
 - Service $\Rightarrow$ Endpoint Auto-Management $\Rightarrow$ Ingress / Istio Real-Time Reference
- ✓ Advantages:
 - Auto-scaling support
 - Automatic failure response
 - Zero-Downtime Deployment

Rewrite

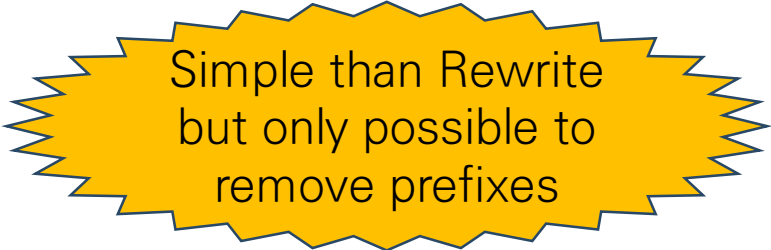
- ✓ Change externally requested path to internal situation
- ✓ For example:
 - External Request: `/api/orders/100`
 - Internal Delivery: `/orders/100`
- ✓ Nginx Example:
 - `rewrite ^/api/(.*)$ /$1 break;`



PrefixStrip

- ✓ Remove front part of url
- ✓ For example:
 - External Request: /api/orders/100
 - Internal Delivery: /orders/100
- ✓ Example:

```
middlewares:  
  strip-api:  
    stripPrefix:  
      prefixes:  
        - /api
```



Simple than Rewrite
but only possible to
remove prefixes

Reasons for Using Rewrite and PrefixStrip

- ✓ External API Standardization
 - External: /api/v1/*
 - Inside: /orders/*
- ✓ Legacy Interworking
 - Older systems: /legacy/*
 - New system: /modern/*
- ✓ Integration of microservices
 - Multiple services can be integrated when using different path schemes

Canary Routing

✓ Gradually deploy the new version to only some users

✓ Concept

– 90% → v1

– 10% → v2

✓ Gradual enlargement:

90/10 ⇒ 70/30

⇒ 50/50 ⇒ 0/100

✓ Goal

- Minimize Disability
- Validating real traffic
- Performance Comparison
- Quick Rollback

State 0



State 1



State 2

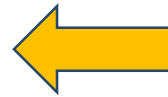


Final State



Canary Routing

```
http:  
- route:  
  - destination:  
    host: order-service  
    subset: v1  
    weight: 90  
  
  - destination:  
    host: order-service  
    subset: v2  
    weight: 10
```



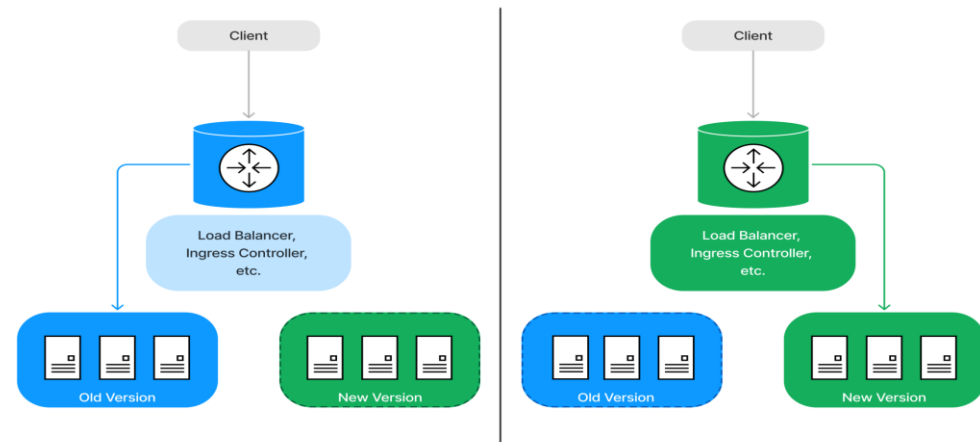
Use Istio as an example

Blue/Green Routing

- ✓ Maintain two complete production environments
- ✓ Structure
 - Blue = Current Operations
 - Green = New Version
- ✓ Transition Moment: Blue → Green
- ✓ Advantages
 - Rollback is possible immediately
 - Switch back to Blue
 - Hardly Downtime
 - Proactively validated in production environments

✓ Weakness

- Cost increase: 2 sets of production environment required
- DB Compatibility Issues: Difficulty Changing Schemas



Canary Blue/Green Routing Comparison

category	Canary	Blue-Green
Traffic Distribution	Only part of it is new	Full Transition
Degree of danger	Very low	Instantaneous
Rollback	Easy	Very easy
Operational Complexity	High	Relatively low
Infrastructure costs	Relatively little	High
Verification for actual use	Very good	Restrictive
Deployment Rate	Gradual	Immediately

2. Authentication & Authorization

- ✓ JWT Token Verification
- ✓ OAuth2 / Open ID Connect (OIDC)
- ✓ API Key-based authentication
- ✓ Role-based Access Control (RBAC)
- ✓ Token Relay (Gateway → Internal Service Delivery)

JWT Token Verification

- ✓ JSON Web Token (JWT) is a signed token containing user authentication information
 - Structure: Header, Payload, Signature
- ✓ JWT Authentication Flow
 - User login $\Rightarrow$ authentication server issues JWT $\Rightarrow$ clients store JWT $\Rightarrow$ API Authorization Header transfer $\Rightarrow$ Gateway/Service verifies JWT
- ✓ JWT Verification Process
 - ① Signature Verification: Is the token tampered with?
 - ② Check Expiration Time (exp): Has the token expired?
 - ③ Issuer check: is it a trusted authentication server?
 - ④ Audience Check: Is this a token for the service?

Working with OAuth2

- ✓ OAuth2 is not an "authentication" but an "authorization" framework
 - Users do not share passwords directly, but delegate external service privileges
 - For example, "Google login", "Kakao login"
- ✓ Components

Components	Role
Resource Owner	User
Client	Application
Authorization Server	Authentication Server
Resource Server	API Server

Working with OAuth2

- ✓ Process
 - User ⇒ Google Login ⇒ Authorization Server ⇒ Access Token Issuance ⇒ Client call API
- ✓ Authorization Code Flow: After logging in to the browser, issue an authentication code and exchange the Access Token.
 - SPA + Backend + Widely used on mobile
- ✓ Feature

Characteristics	Description
External login is possible	Google/Kakao/GitHub
SSO enabled	Single Sign-On
Token-based	Stateless
delegation of authority	API Access Control

OpenID Connect (OIDC)

- ✓ OIDC is the addition of the "Authentication" function on top of OAuth2
- ✓ OAuth2 + User Authentication Standard

Additional Elements	Purpose
ID Token	User identity
UserInfo Endpoint	Standard User Information
subject(sub)	User-specific ID
issuer(iss)	Identifying the Authentication Server
audience(aud)	Token destination
nonce	Prevent replay
auth_time	Login Time

OpenID Connect (OIDC)

- ✓ OIDC provides ID Token in JWT format
- ✓ OIDC Login Flow

- User Login $\Rightarrow$ Authorization Server $\Rightarrow$ Access Token + ID Token Issuing $\Rightarrow$ Client Authentication Completed

- ✓ Representative OIDC Provider
 - Google, Keycloak, Auth0, Okta

OIDC

- OAuth2 Authorization Code Flow
- OAuth2 Access Token
- OAuth2 Refresh Token
- + ID Token

category	OAuth2	OIDC
Purpose	delegation of authority	User Authentication
User information	No standard	ID Token Provision
Login Features	restrictive	Official Support

API Key-based authentication

- ✓ simplest authentication method
- ✓ Structure:
 - Client delivers Key, like as X-API-KEY: abc123
 - Authorization with ApiKey, abc123
- ✓ API Key Inquiry: confirm validation $\Rightarrow$ allow requests
- ✓ Advantages
 - Easy to implement and fast
- ✓ Weakness
 - It's hard to know who called, and it's possible to leak a key

Role-based Access Control (RBAC)

- ✓ Restrict access based on user role
 - USER, ADMIN, MANAGER, OPERATOR
- ✓ Combined with JWT

```
{  
  "sub": "user1",  
  "roles": ["ADMIN", "USER"]  
}
```

API	right
GET /products	USER
POST /products	ADMIN
DELETE /users	SUPER_ADMIN

Token Relay

- ✓ Method of delivering JWT received by Gateway to internal service as it is
 - Client $\Rightarrow$ API Gateway $\Rightarrow$ Order Service $\Rightarrow$ Inventory Service
- ✓ Internal services do not know user information if only Gateway is authenticated
- ✓ Method: Maintain Authorization Header
 - Authorization: Bearer eyJ...
 - Gateway forward the same header

3. Request/Response Transformation

- ✓ Add/Delete Request Header
- ✓ Response Header process
- ✓ Body Transformation (JSON Structural Change)
- ✓ API Aggregation (integration of multiple service responses)

Add/Delete Request Header

- ✓ The ability of the API Gateway to modify HTTP Header before forwarding a request to an internal service
- ✓ Add Header Example:
 - X-Request-Id: abc123
 - X-User-Id: user1
 - X-Correlation-Id: order578
- ✓ Additional reason
 - Forwarding user information. Distributed tracking. Internal authentication
- ✓ Reason for deletion
 - Remove sensitive information
 - Header Collision Prevention

Response Header process

- ✓ The ability of the gateway to modify the Response Header (mainly a security issue)
- ✓ Add Security Header
 - X-Frame-Options: DENY
 - X-XSS-Protection: 1; mode=block
- ✓ Prevent browser attacks
- ✓ CORS PROCESSING
 - e.g. : Access-Control-Allow-Origin: https://frontend.com
- ✓ Cache Policy
- ✓ Remove Header
 - Hide internal server information

API Aggregation

- ✓ Combining multiple internal service responses into one
- ✓ Problem situation: Client needs to call order service, user service, product service, delivery service all to configure order inquiry screen
 - heavy calls, network growth, mobile inefficiencies, client complexity
- ✓ Client $\Rightarrow$ API Gateway / BFF $\Rightarrow$ Internal service Multiple call $\Rightarrow$ responses returned in combination

Location	Description
API Gateway	Simple Aggregation
BFF(Backend For Frontend)	UI Customization Aggregation
GraphQL	Query Based Aggregation
Dedicated Aggregator Service	Complex combination processing

4. Rate Limiting / Throttling

- ✓ IP-based request limitation
- ✓ User/Token-based restrictions
- ✓ Burst Control
- ✓ Redis-based Distributed Rate Limit

IP-based request limitation

- ✓ Limiting request number with client IP
- ✓ Example
 - 192.168.0.1 → 100 requests per minute allowed
 - Over 100 requests: Too Many Requests issued (HTTP 429)
- ✓ Process
 - Count requests ⇒ Check if limits are exceeded ⇒ Allow or block

IP-based request limitation

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: delivery-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/limit-rpm: "100"
    nginx.ingress.kubernetes.io/configuration-snippet: |
      limit_req_status 429;
spec:
  ingressClassName: nginx
  rules:
    - host: delivery.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: delivery-keycloak
                port:
                  number: 80
```

User/Token-based restrictions

- ✓ limit with JWT/User/API Key
- ✓ Example
 - userId=user1 → 1000 requests per minute
 - API Key abc123 → 50 requests per second
- ✓ Advantages
 - User-level control
 - Multi-tenant support
 - API commercialization

Burst Control

- ✓ Ability to reduce instantaneous traffic bursts
 - Allow a moment burst while maintaining the average limit
- ✓ Burst Case
 - Average of 100 req/minutes
 - Burst 500 allowed
- ✓ Token Bucket Algorithm: Representative
 - The token is in the bucket
 - Consume token on request
 - Token recharge at over time

```
nginx.ingress.kubernetes.io/limit-rpm: "100"  
nginx.ingress.kubernetes.io/limit-burst-  
multiplier: "5"
```

Redis-based Distributed Rate Limit

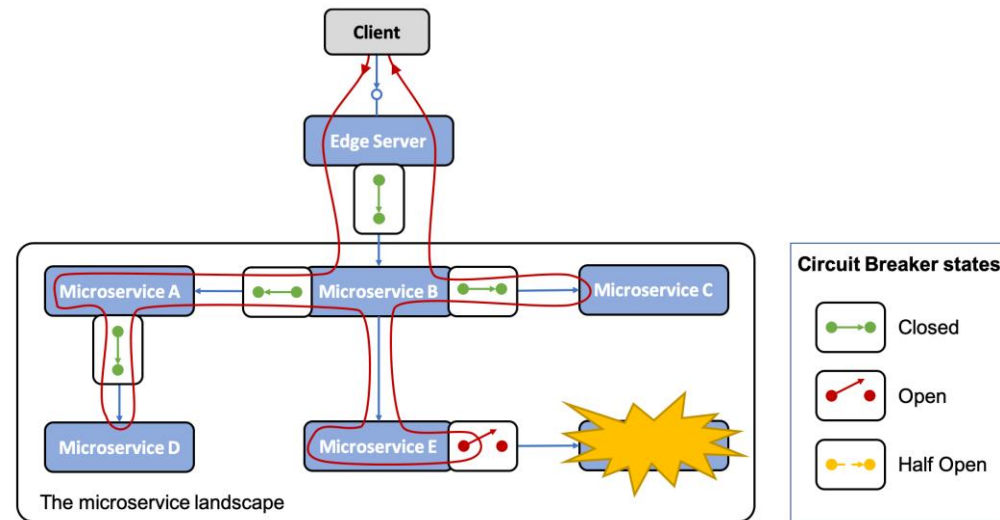
- ✓ MSA/Kubernetes has multiple servers, resulting in discrepancies in the counters per server
- ✓ Example:
 - Gateway A → 10 times allowed, Gateway B → 10 times allowed
 - Result: 20 allowed in practice
- ✓ Solution plan
 - Managing Common Counters with Redis

5. Circuit Breaker

- ✓ Block request when certain services fail
- ✓ Fail Fast Strategy
- ✓ Provide Fallback Response
- ✓ Connect Resilience4j

Block request when certain services fail

- ✓ Cascading Failure prevent
- ✓ Circuit Breaker state
 - CLOSED: Normal state
- ✓ OPEN:
 - Disability status
 - Request failed immediately
 - Do not call the actual service
- ✓ HALF-OPEN
 - Recovery Verification Status
 - Test only some requests



Fail Fast Strategy

- ✓ Strategy for immediate failure without waiting too long for a problematic service
- ✓ A problem situation
 - Service timeout 30 seconds $\Rightarrow$ Thread Occupancy $\Rightarrow$ Connection Occupancy $\Rightarrow$ user response delay
 - Failed processing in 100 ms
- ✓ Use with Timeout
 - Timeout + Circuit Breaker
 - Example: 2 second timeout, 5 failed  Circuit OPEN

Provide Fallback Response

- ✓ Provide an alternative response in case of failure
 - For example, return the default popular product if the recommended product inquiry fails
 - Client $\Rightarrow$ Product Service $\Rightarrow$ Recommendation Service (Disorder)
 $\Rightarrow$ Fallback: Popular Products Return
- ✓ Fallback Type
 - Static Fallback: Fixed Response
 - Cached Fallback: Return cache data, using Redis Cache
 - Default Business Response: Basic Business Processing
Example: Shipping cost calculation failed $\rightarrow$ Use basic shipping cost
- ✓ Fallback is fault-hiding technology; log, monitoring, fault detection required

Resilience4j connect

- ✓ The most representative Fault Tolerance library in Java/Spring Boot

Function	Description
CircuitBreaker	Disability Blocking
Retry	Retry
RateLimiter	Request restrictions
Bulkhead	isolation
Timeout	Time limit
Cache	Caching Results

Resilience4j connect

```
@CircuitBreaker(  
    name = "paymentService",  
    fallbackMethod = "fallback"  
)  
public Payment getPayment() {  
    return paymentClient.call();  
}
```

```
public Payment fallback(Exception e) {  
    return new Payment("temporary unavailable");  
}
```

```
resilience4j:  
  circuitbreaker:  
    instances:  
      paymentService:  
        failureRateThreshold: 50  
        waitDurationInOpenState: 10s  
        slidingWindowSize: 10
```

- Circuit OPEN when the failure rate is higher than 50%
- Circuit Open State Retention Time
- Calculate failure rates based on the last 10 requests

7. Retry / Timeout Control

- ✓ Automatic Retry Policy
- ✓ Timeout settings (by service)
- ✓ Exponential Backoff: Adjust the number of retries by increasing the retries time interval to 2^n form
- ✓ Prevention of Disruptions

8. Logging / Monitoring / Tracing

- ✓ Collect Request/Response Logs
- ✓ Create and forward Correlation IDs
- ✓ Distributed Tracing (Zipkin, Jaeger)
- ✓ Metrics (Prometheus)

9. Security

- ✓ SSL/TLS Termination
- ✓ CORS 정책 적용 (CORS: Cross Origin Resource Sharing)
- ✓ IP Whitelisting / Blacklisting
- ✓ WAF interworking is possible

API G/W Key Technologies

skill	type	a key feature	advantage	Weakness	Representative application
NGINX / NGINX Ingress Controller	Reverse Proxy / Ingress	Most widely used L7 proxy	Good performance, simple setup, Kubernetes standard	API management is relatively weak	Kubernetes Ingress, simple API Gateway
Kong	API Gateway	NGINX-based API Gateway	Plugin rich, OAuth2/OIDC strong, easy to manage	Large-scale DB/Plugin management complexity	SaaS, MSA API Gateway
Apache APISIX	API Gateway	etcd-based dynamic gateways	High performance, real-time settings change, Plugin strong	Ecosystem is smaller than Kong	Cloud Native API Gateway
Spring Cloud Gateway	Java Reactive Gateway	Spring WebFlux-based	Excellent Spring Ecosystem Integration	Java memory usage high	Spring Boot MSA
Istio IngressGateway	Service Mesh Edge Gateway	Envoy-based Mesh Gateway	mTLS, Traffic Split, Canary Powerful	High operational difficulty	Large-scale Kubernetes MSA
Traefik	Cloud Native Proxy	Docker/K8s Friendly Auto Settings	Very Easy Settings	Lack of policy functionality at the enterprise level	Small Kubernetes

API G/W Key Technologies

skill	type	a key feature	advantage	Weakness	Representative application
HAProxy	L4/L7 Load Balancer	High Performance TCP/HTTP Proxy	Very fast, very stable	Management UI/API Relatively Weak	Financial / High Performance Systems
Envoy	Cloud Native Proxy	CNCF Standard Proxy	xDS-based dynamic setup, Mesh core	Direct setup is complicated	Istio/App Mesh-based
Amazon API Gateway	Managed Gateway	AWS Fully Managed	No server operations required	Vendor dependency, cost increase	AWS Serverless
Apigee	Enterprise API Management	Google Enterprise API Management	Analysis/Policy /Portal Powerful	Expensive, heavy	Large enterprise API platform
WSO2 API Manager	Enterprise API Management	Open-source Enterprise Gateway	IAM/OAuth Powerful	Configuration Complexity	Public/Financial API
Tyk	API Gateway	Go-based Gateway	Light and fast	Ecosystems Relatively Small	Small and Medium API Platform

NGINX Ingress Controller

✓ Characteristics

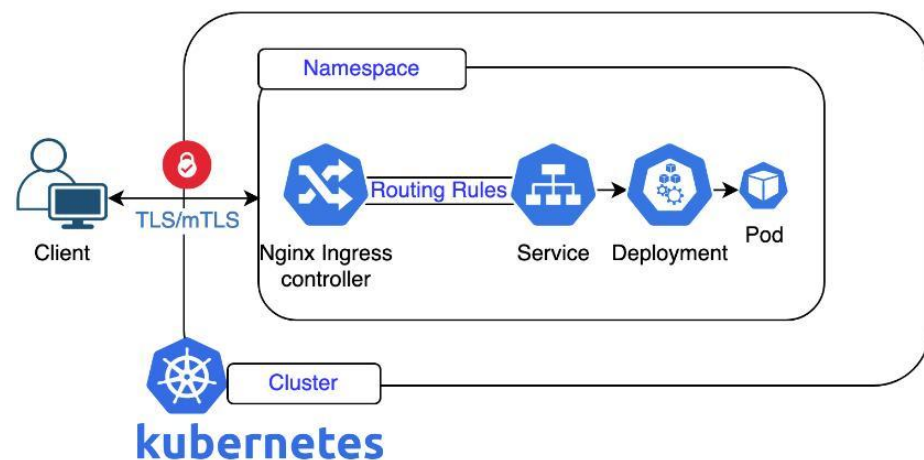
- Kubernetes Default Ingress Implementation
- Default API Gateway role in effect
- NGINX-based (very reliable)

✓ Role

- External → Internal Service Routing
- TLS Termination
- Default CORS/Rate Limit

✓ Advantages

- Easy to install (kubectl application)
- Many documents/references



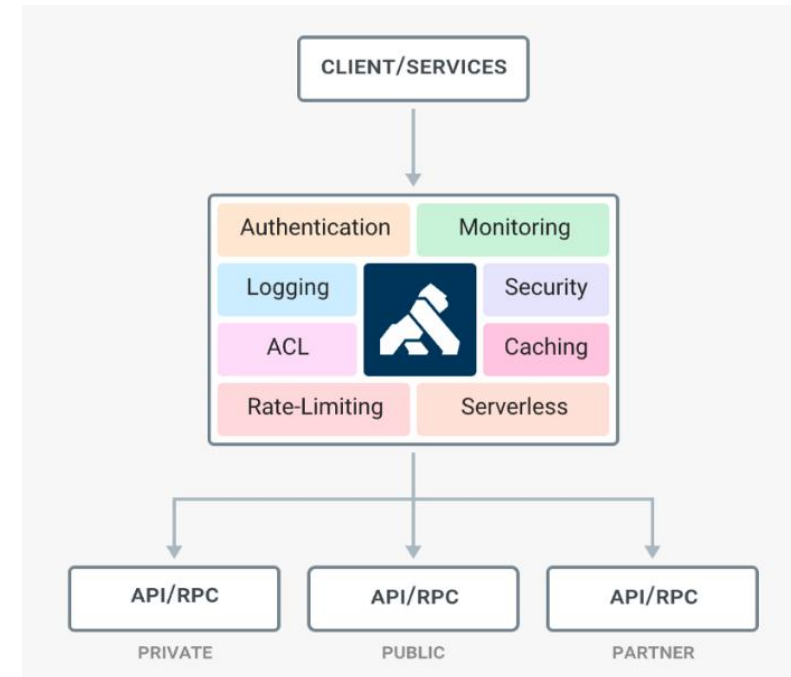
Kubernetes deployments diagram

Let's get started!

Deploying Nainx Ingress Controller to a Kubernetes cluster

Kong Ingress Controller

- ✓ Characteristics
 - API Gateway Only Products
 - Plugin-based (authentication, logging, rate limit, etc.)
- ✓ Advantages
 - Strong API management capabilities
 - DevPortal, API Key, policy manageable
 - Enterprise API Platform
- ✓ application
 - It's not just routing,
 - When you need an API management platform



MSA Advanced Structure (Service Mesh): Istio + Gateway

✓ Characteristics

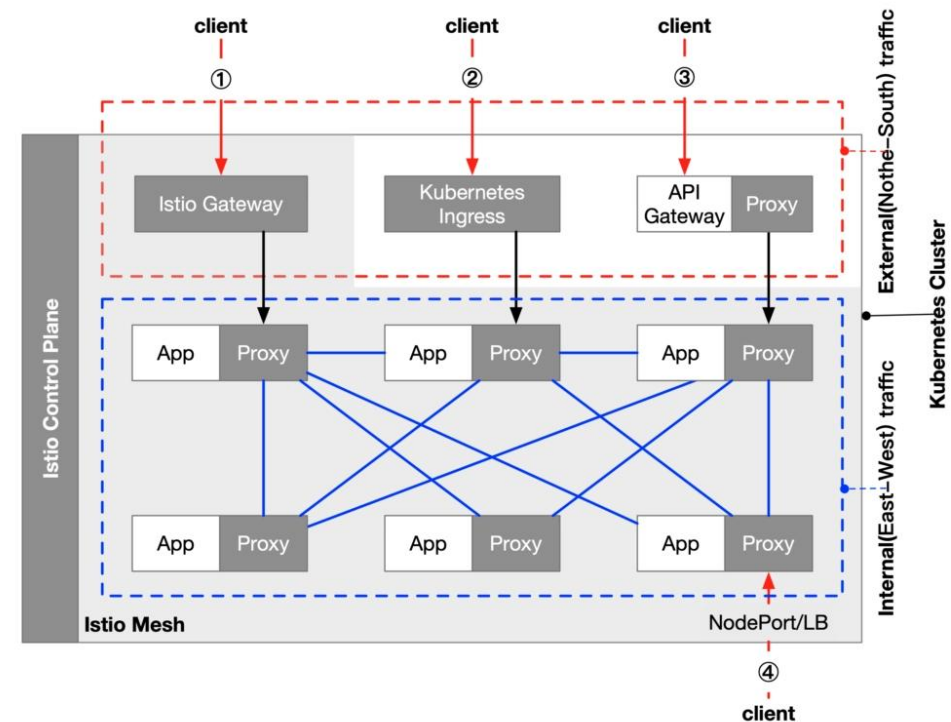
- Based on Envoy Proxy
- Gateway + Sidecar Structure

✓ Function

- Traffic Control (A/B Test, Canary)
- Control communication between mTLS security services

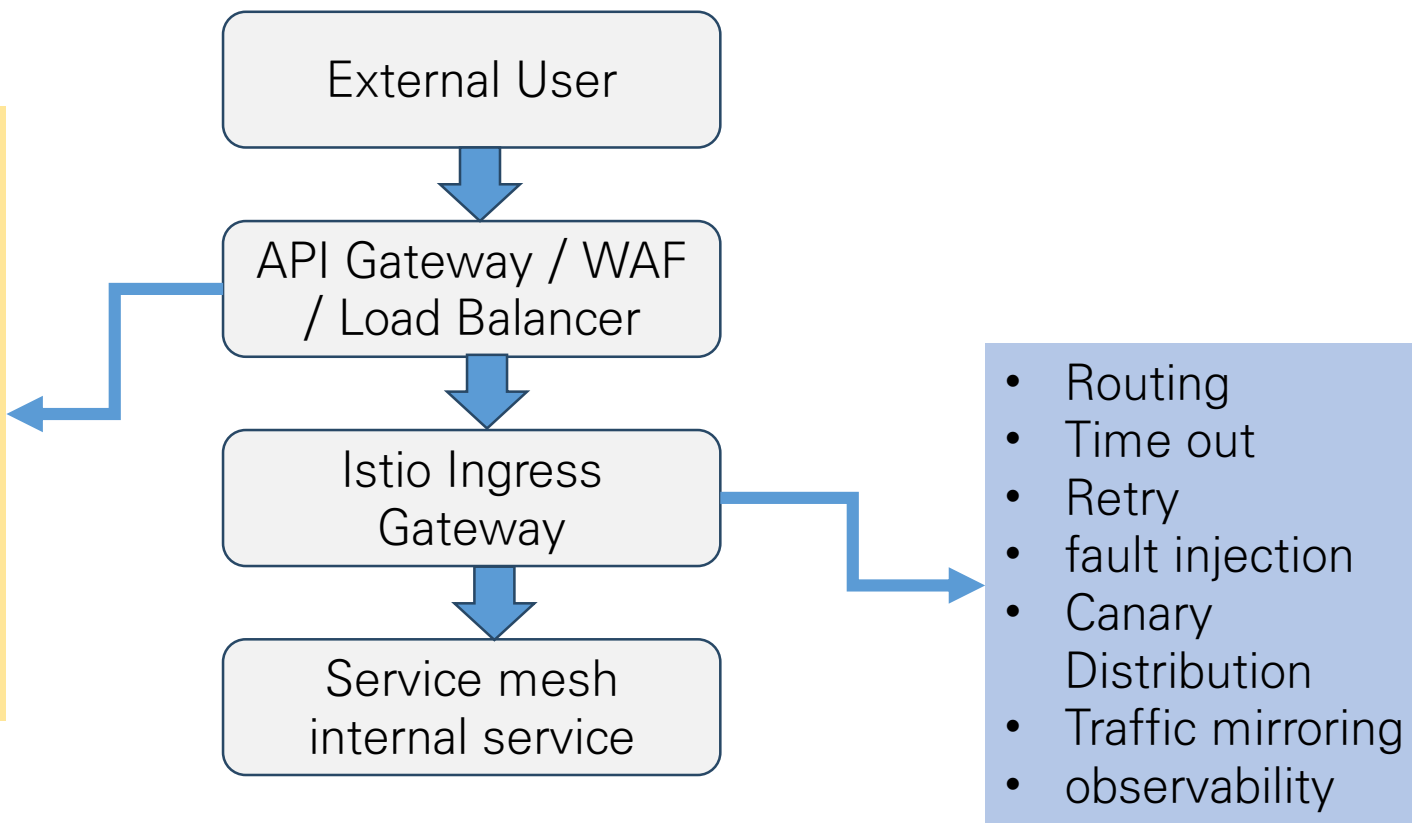
✓ When do you use it?

- large-scale MSA
- Traffic Control/observability when it's important



Practical point of view

- API authentication,
- API Key,
- Rate Limit,
- Developer Portal,
- Charges,
- Managing external customer APIs



Pros

- ✓ Centralize common features
- ✓ Enhanced Security
- ✓ Improve service independence
- ✓ Simplifying the Client

Cons

- ✓ Single point of failure (SPOF)
- ✓ Additional network hops
- ✓ Possible performance bottlenecks
- ✓ Increasing operational complexity

Consideration

- ✓ Gateway Redundancy (HA)
- ✓ Select authentication method
- ✓ Rate Limiting strategy
- ✓ Circuit Breaker
- ✓ Logs/monitoring schemes

Precautions

- ✓ Ban excessive logic on Gateway
- ✓ Business logic is located in the service
- ✓ Bottleneck point management required
- ✓ timeout / retry policy required

Installing Ingress

- ✓ `kubectl apply -f`
`https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml`

Confirmation of installation

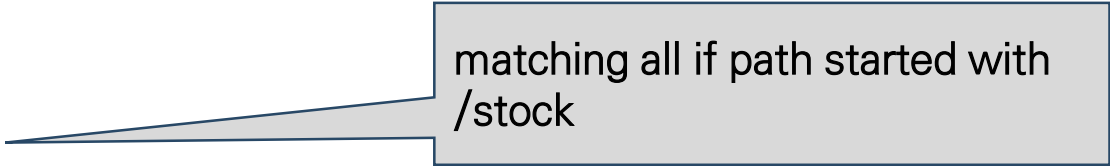
- ✓ `kubectl get pods -n ingress-nginx`
- ✓ `kubectl get svc -n ingress-nginx`

To delete

- ✓ `kubectl delete namespace ingress-nginx`

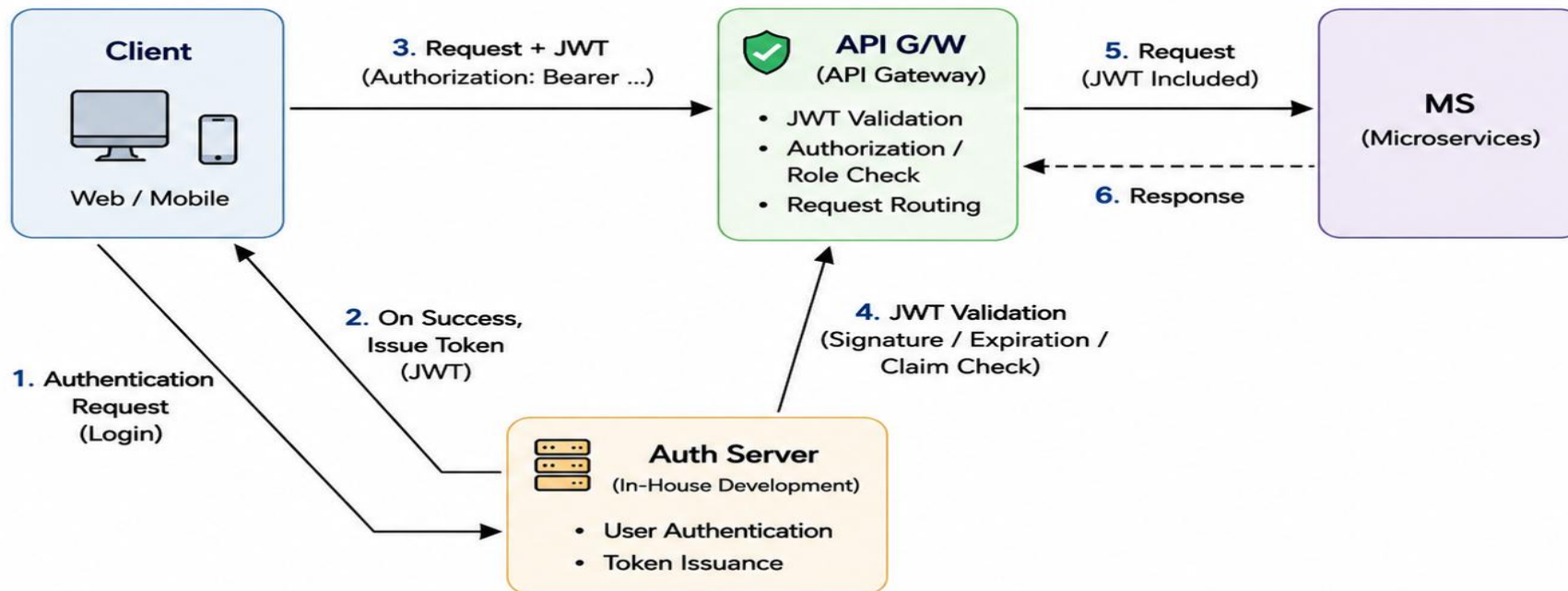
Creating an Ingress yaml file: general situation

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: stockconfirm-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
spec:
  ingressClassName: nginx
  rules:
    - host: localhost
      http:
        paths:
          - path: /stock
            pathType: Prefix
            backend:
              service:
                name: stockconfirm-service
                port:
                  number: 80
```



matching all if path started with
/stock

Ingress, JWT Practice

**1. Authentication Request :**

The client requests login from the Auth Server.

2. On Success, Issue Token :

The Auth Server issues a JWT token upon successful authentication.

3. Request + JWT :

The client sends a request to the API Gateway with the JWT token.

4. JWT Validation :

The API Gateway validates the JWT for signature, expiration, and claims.

5. Request Forwarding :

After validation and authorization, the API Gateway forwards the request to the corresponding MS.

6. Response :

The MS processes the request and returns the response to the client via the API Gateway.

Authentication and OAuth2 with Keycloak

- ✓ Highly popular open-source identity and access management (IAM) solution
- ✓ IAM server for centralized – integrated management of user authentication and authorization
 - OIDC / OAuth2/SAML Standard Support
 - SSO(Single Sign-On)
 - Issuance of JWT
 - Social login link
 - Role-based privileges (RBAC)
 - Provide User/Group Management UI
 - Easy to work with Spring Security
 - Combine with Kubernetes/Ingress/Istio

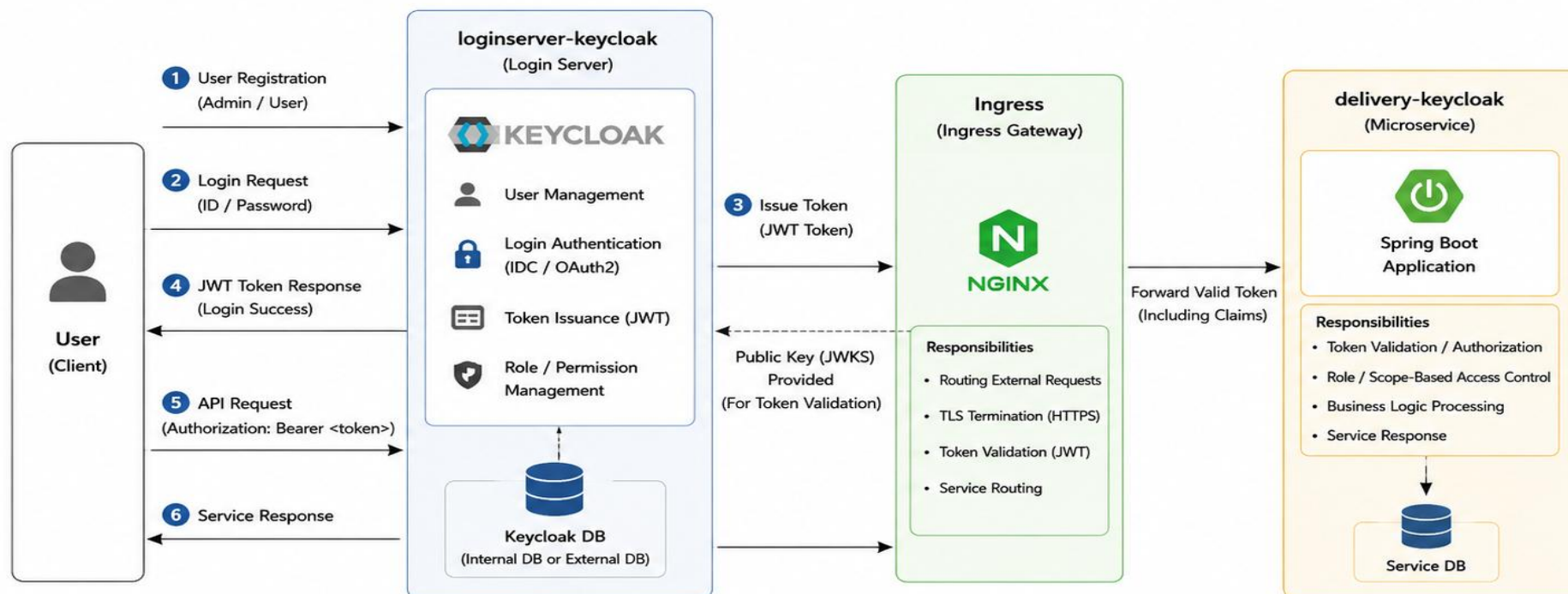
Authentication procedures with KeyCloak

1. Kubernetes Keycloak install : id admin, password admin123
2. Create Ingress for Keycloak External Connections
3. Create realm: secret
4. Client create : loginserver-keycloak
5. Create user
6. Issuance of token
7. delivery-service oauth2-resource-server settings
8. Call delivery to Ingress

API G/W Implementation Practice

Overall Flow

1. User Registration → 2. Login / Authentication Request → 3. Issue Token → 4. Forward Token → 5. Access Service via Ingress → 6. Service Response



Component Description

- loginserver-keycloak** : User registration, authentication, and token (JWT) issuance
- Ingress** : Handles external requests, validates JWT tokens, and routes to services
- delivery-keycloak** : Validates tokens and processes business logic
- User** : Requests APIs after login and receives token for authentication

Token Validation Flow

- Keycloak issues JWT token (including claims)
- Ingress validates JWT token (using JWKS public key)
- If validation succeeds, the request is forwarded to the backend service
- The service verifies required permissions/roles and processes the request

Keycloak Structure

✓ Realm

- Fully Independent Certification Zones
- User Set
- Client Set
- Authority system
- Top unit to bind a login policy

Realm

```
graph LR; Realm --- Users; Realm --- Clients; Realm --- Roles; Realm --- Groups; Realm --- IdentityProviders[Identity Providers]; Realm --- Authentication; Realm --- Tokens;
```

✓ If Realm is different:

- User not shared
- Token not shared
- Log-in is also independent
- Authority independent

Keycloak Structure

- ✓ Client: Applications with Keycloak
 - e.g : Spring Boot API, Android App, Gateway
- ✓ Client ID: Identifier of the Client
 - e.g : client-id = loginserver-keycloak
- ✓ User: login user
 - Username: Login ID entered by the user
 - User ID: Keycloak internal UUID (PK)
- ✓ Realm Role: Realm All Common Privileges
- ✓ Client Role: Specific Client-only privileges.
 - delivery-keycloak: DELIVERY_ADMIN, DELIVERY_USER

Keycloak Structure

- ✓ Token: Keycloak Key Features
 - Upon successful login, issue JWT Access Token
- ✓ Access Token: Token for API calls
 - e,g : Authorization: Bearer eyJhb...
- ✓ Refresh Token: When Access Token expires, for issuing new tokens without re-logging

token	JWT or not	the main purpose
Access Token	Most JWT	API authentication
ID Token	Nearly JWT	Login User Information
Refresh Token	Weather JWT or not	Reissue token

Keycloak token classification

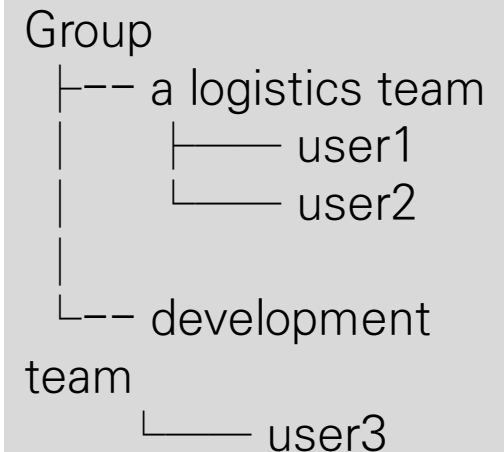
- ✓ Authentication: Check who the user is, i.e. login
- ✓ Authorization: "See what you can do"
 - Example: Only ADMIN can delete, USER can review
- ✓ Required Actions: Forced Action on User First Login
 - Example: Verify Email, Update Password, Configure OTP, Update Profile

Keycloak Structure

- ✓ Identity Provider (IDP): External login interworking
 - Google, GitHub, Naver, Kakao
- ✓ Federation: Interworking external user storage
 - LDAP, Active Directory
 - Allows you to log in to your company account as it is
- ✓ OIDC: Keycloak Default Core Protocol.
 - OpenID Connect, OAuth2-based login standard
- ✓ OAuth2: Empowerment Standard
 - Example: Google login, GitHub login
- ✓ Service Account: Used to authenticate between services without users
 - delivery-service ↔ inventory-service

Keycloak Structure

- ✓ Group: User Set (Organization/Department/Category)
 - Features that logically bind users
 - If you give permission to each individual user, it's difficult to manage
- ✓ Session : Current login status
 - Authentication status maintained after the user logs in successfully
 - Keycloak Session: Login user, login time, IP, Client, Refresh Token status



Keycloak Structure

- ✓ Event : Record security/authentication activity that occurred inside the Keycloak
- ✓ Similar to Audit Log
- ✓ Track login failures, audit password changes, detect anomalies, and track multiple login failures in a short time
 - 예: LOGIN, LOGIN_ERROR, LOGOUT, REGISTER, UPDATE_PASSWORD, REFRESH_TOKEN

Authentication and OAuth2 with KeyCloak

- ✓ client create client auth⇒ON, Standard flow ⇒ ON, direct access ⇒ ON

Clients > Create client

Create client

Clients are applications and services that can request authentication of a user.

1 General settings

2 Capability config

3 Login settings

Client authentication  On

Authorization  Off

Authentication flow

☒ Standard flow

☒ Direct access grants

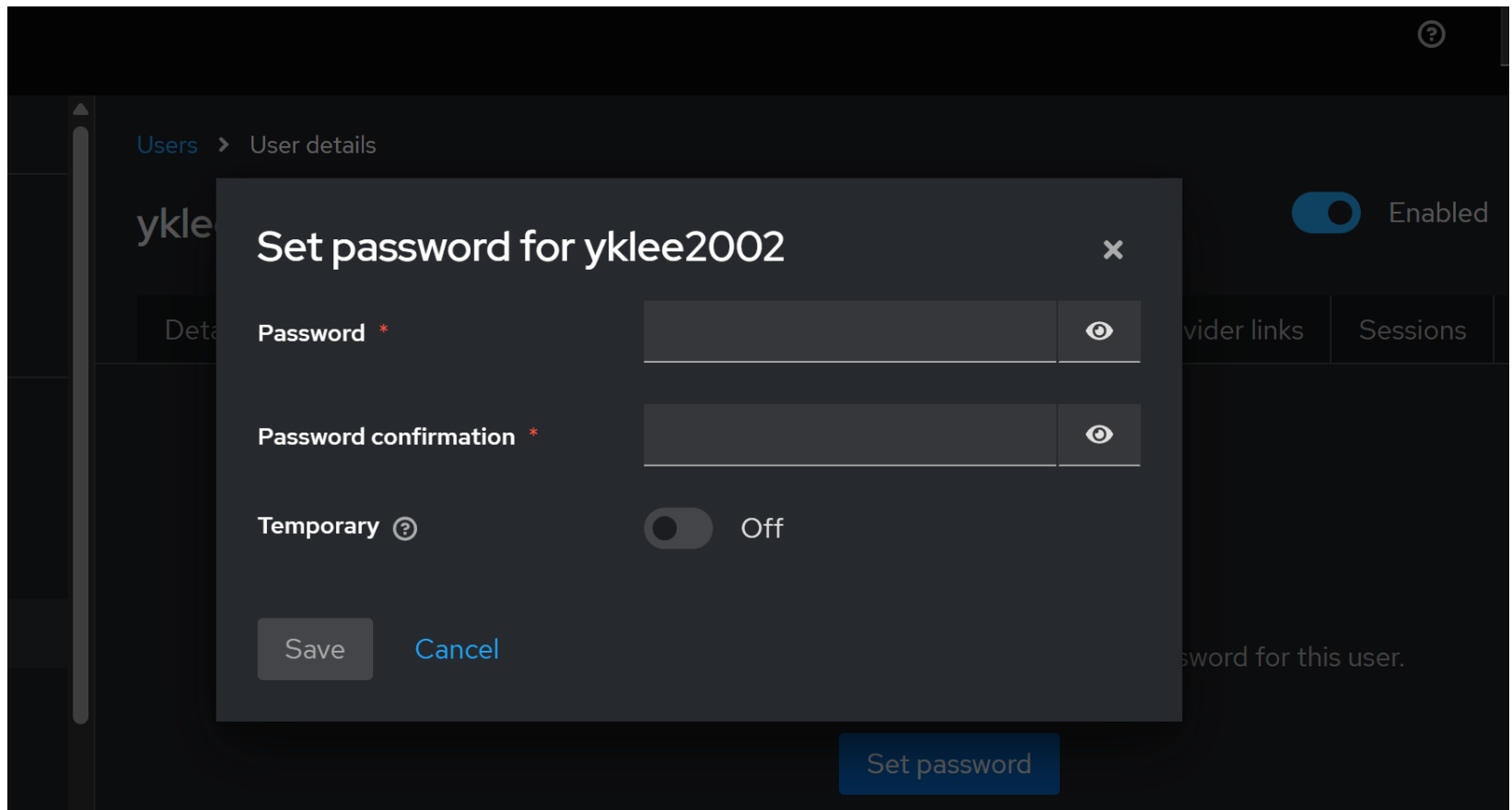
☐ Implicit flow

☐ Standard Token Exchange

☐ Service accounts roles

Authentication and OAuth2 with KeyCloak

- ✓ User password must be temporarily turned off when writing it



Authentication and OAuth2 with KeyCloak

✓ delivery service Settings

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>  
</dependency>
```

✓ application.properties Settings

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=  
http://keycloak.keycloak.svc.cluster.local:8080/realms/cretec
```